omplete Project Specification

What this project is

Dynamic Shortest Path Analysis on Probabilistic Graphs with Adversarial Edge Updates

Standard dynamic Dijkstra is textbook. Your version extends it to edges with uncertainty (mean weight + variance), adversarial update sequences that maximize disruption, and a live visualizer showing exactly which nodes get re-evaluated. No undergraduate project does this combination.

Full Feature List

Backend (C++)

1. Custom binary min-heap — insert, extract-min, decrease-key, heapify. No STL priority queue.
2. Graph representation — adjacency list, each edge stores (weight, sigma) pair.
3. Standard Dijkstra — baseline, deterministic weights.
4. Probabilistic Dijkstra — cost = $w + k \cdot \sigma$, where k is a configurable risk factor. Risk-averse vs cost-optimized routing toggle.
5. Bellman-Ford — handles negative weights, used for comparison table.
6. SPT (Shortest Path Tree) maintenance — tracks which edges are in the current SPT.
7. Selective recomputation — if updated edge is outside SPT, skip. If inside, partial recompute only from affected subtree.
8. Full recomputation — baseline comparison mode.
9. Single edge update — add, remove, change weight, change sigma.
10. Batch update handler — multiple simultaneous edge updates, conflict detection, resolves before recomputing. $O(k \log n)$ vs naive $O(k \times n)$.
11. Adversarial update generator — finds the SPT edge whose update maximally disrupts current path. Simulates an adversary.
12. Non-adversarial (random) update generator — random edge, random new weight. Baseline comparison.
13. Experimental evaluator — runs both modes, logs nodes re-evaluated, time taken, path cost difference.

14. JSON output layer — every algorithm event serialized to JSON, piped to Flask.

Backend (Python Flask)

15. REST API — /run, /update, /batch, /reset endpoints.
16. WebSocket server — streams algorithm events to frontend in real time.
17. OpenStreetMap loader — OSMNX pulls Noida road network, maps to your graph format.
18. Experiment runner — automated benchmark comparing all three algorithms across graph types.

Frontend (React + D3)

19. Force-directed graph layout — nodes repel, edges act as springs. Graph breathes and settles naturally.
20. SPT edge highlighting — purple glow on SPT edges, teal trace on optimal source→target path.
21. Node re-evaluation animation — ripple effect from update point, nodes flash as they get re-evaluated. Stops early in selective mode — visually proves the optimization.
22. Edge uncertainty visualization — dashed edges for high σ, edge thickness proportional to variance, color shifts red as uncertainty grows.
23. Adversarial vs non-adversarial toggle — side-by-side or switchable view, both modes running on same graph.
24. Algorithm comparison panel — live bars for Dijkstra+σ, standard Dijkstra, Bellman-Ford. Updates after every run.
25. Risk factor slider (k·σ) — drag to change how much uncertainty penalizes routing. Path changes live.
26. Source/target node selector — click any node to set.
27. Selective vs full recompute toggle — switches backend mode, comparison logged.
28. Adversarial attack button — triggers worst-case edge disruption with animation.
29. Batch update button — 3 simultaneous edge updates, conflict detection logged.
30. Node hover tooltip — dist, σ, predecessor, re-eval count.
31. Real-time event log — every update, recompute, attack timestamped.
32. Stats panel — nodes, edges, total updates, total re-evaluations.
33. Animation speed control.

34. OpenStreetMap mode — switch from synthetic graph to Noida road network.

Stack

- C++ (algorithm core, compiled separately)
- Python Flask + Flask-SocketIO (bridge)
- React + D3.js (frontend)
- OSMNX (real-world graph data)
- GitHub Copilot in VS Code (your build tool)

File Structure

```
project/
 ├── cpp/
 |   ├── graph.h        # Node, Edge structs with sigma
 |   ├── heap.h         # Custom min-heap
 |   ├── dijkstra.cpp   # Standard + probabilistic
 |   ├── bellman.cpp    # Bellman-Ford
 |   ├── spt.cpp        # SPT maintenance + selective recompute
 |   ├── adversarial.cpp  # Adversarial + random update generators
 |   ├── batch.cpp      # Batch update + conflict detection
 |   └── main.cpp       # JSON output, stdin command loop
 ├── server/
 |   ├── app.py         # Flask + SocketIO
 |   ├── bridge.py      # Subprocess manager for C++ binary
 |   └── osm_loader.py  # OpenStreetMap → graph format
 ├── client/
 |   ├── src/
 |   |   ├── App.jsx
```

```
|   |   ├── GraphCanvas.jsx    # D3 force layout + animation
|   |   ├── SidePanel.jsx      # Algo comparison + stats + log
|   |   ├── Controls.jsx       # All buttons + sliders
|   |   └── useSocket.js       # WebSocket hook
|   └── package.json
└── data/
    └── noida_graph.json   # Preprocessed OSM data
```